

1 Esercizio 1

```
int A=0;
int B=0;
```

```
Process P
while(true){
p1: Sezione Non Critica
p2: A=1;
p3: while(B!=0){}
p4: Sezione Critica
p5: A=0;
}
```

```
Process Q
while(true){
q1: Sezione Non Critica
q2: while(A!=0){}
q3: B=1;
q4: Sezione Critica
q5: B=0;
}
```

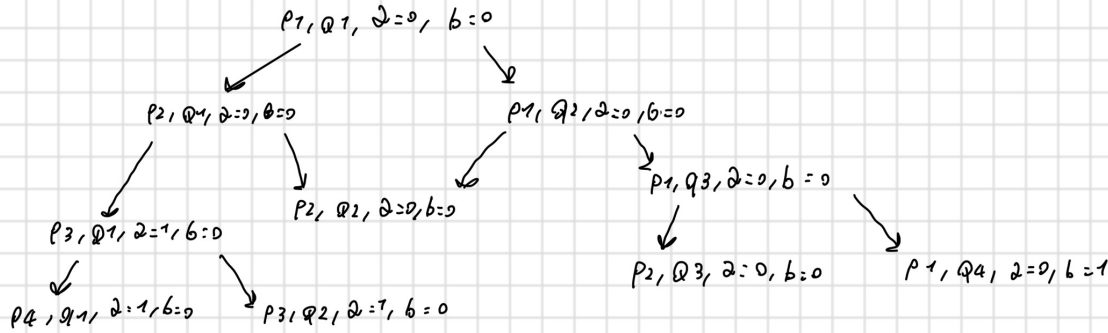
Figura 1: Algoritmo di mutua esclusione

Consideriamo l'algoritmo di mutua esclusione proposto alla figura 1.

1. Disegnare l'inizio dello diagramma degli stati per questo algoritmo (massimo 10 stati).
2. Questo algoritmo verifica la proprietà di mutua esclusione? Giustificare.
3. Questo algoritmo verifica la proprietà d'assenza di deadlock? Giustificare.
4. Questo algoritmo verifica la proprietà d'assenza di starvation? Giustificare.

Esercizio 1

1)



2) L'algoritmo NON garantisce mutua esclusione. Se eseguiamo il processo Q fino a q2, abbiamo A=B=0, quindi possiamo andare a q3. A questo punto

se viene data precedenza a P, può procedere fino a SC, appena entra in SC Q può continuare fino a sezione critica a sua volta.

3) Garantisce assenza di deadlock. Per avere un deadlock entrambi devono essere bloccati nel while. Se siamo in P3 allora A=1, allora Q non è bloccato nel while. Quindi B=0 e P esce dal while.

4) No, l'algoritmo non garantisce la proprietà di assenza di starvation. Abbiamo un problema di dead starvation. Se il processo P è molto più

veloce di Q, potrebbe accadere: 1) P esce dalla sezione critica e pone A=0 (p5) 2) prima che Q riesca a valutare la condizione while (A!=0), P riacquista il critico, esegue la sezione non critica e pone di nuovo A=1 (p2) 3) Q esegue il while, vede A!=0 e continua a stare in q2

Esercizio 2

2 Esercizio 2

Consideriamo un sistema fatto di due recipienti A e B e dei produttori e dei consumatori. Il recipiente A è di capacità a e il recipiente B è di capacità b dove a e b sono degli interi strettamente superiori a 0. I produttori eseguono in ciclo le azioni seguente:

- Se A e B sono pieni, aspettano.
- Se A è pieno ma non B, aggiungono un elemento a B altrimenti aggiungono un elemento a A.

I consumatori eseguono in ciclo le azioni seguente:

- Se A e B sono vuoti, aspettano.
- Se B non è vuoto, ritirano un elemento di B altrimenti ritirano un elemento di A.

Proponete in JAVA, C o pseudo-codice una modellazione di questo sistema, prendendo in considerazione che vogliamo un processo per produttore e un processo per consumatore, che prenderemo tre produttore e tre consumatore e che vogliamo evitare le attese attive. Per facilitare il lavoro, potete assumere che i recipienti sono rappresentati da variabili interi condivisi (produrre aumenta di 1 il valore e consumare lo sminuisce di 1).

```

VOID *PRODUCE(VOID *PT) {
    WHILE (TRUE) {
        PTHREAD_MUTEX_LOCK(&LOCK);
        // SE A E B SONO ENTRATI PRIMA, LE PASSIONE ALTRA
        WHILE (A == CAPA || B == CAPB) { 4 CAPA E CAPB SONO LE LORO CAPACITA'
            PTHREAD_COND_WAIT(&COND_PROD, &MUTEX);
        }
    }
}

```

```

// SE A E' PIENO MA NON B, ACCENDO B
IF (A == CAPA || B < CAPB) {
    B++;
} ELSE {
    A++;
}

PTHREAD_COND_SIGNAL(&COND_CONSUME); // SVEGLIA UN CONSUMATORE
PTHREAD_MUTEX_UNLOCK(&LOCK);
}
return NULL;

```

```

VOID *CONSUME(VOID *PT) {
    WHILE (TRUE) {
        PTHREAD_MUTEX_LOCK(&LOCK);
        WHILE (A == 0 || B == 0) {
            PTHREAD_COND_WAIT(&COND_CONSUME, &LOCK);
        }
        IF (B > 0) {
            B--;
        } ELSE {
            A--;
        }
        PTHREAD_COND_SIGNAL(&COND_PROD);
        PTHREAD_MUTEX_UNLOCK(&LOCK);
    }
    return NULL;
}

```

Esercizio 3

Consideriamo l'algoritmo con message passing proposto sulla figura 2. Lo scopo di questo algoritmo è di garantire la mutua esclusione per due processi.

1. Descrivere in qualche righe come funziona questo algoritmo.
2. Questo algoritmo verifica la mutua esclusione? Giustificare.
3. Questo algoritmo verifica l'assenza di starvation? Giustificare e nel caso di una risposta negativa, proporre un'ipotesi per avere l'assenza di starvation.
4. Come modificare l'algoritmo per avere un algoritmo di mutua esclusione per 3 processi (lavorando con l'ipotesi trovata nella domanda precedente)?

```

Process Pi per i uguale a 0 o 1
boolean B0=true; //variabile di P0
boolean B1=false; //variabile di P1

while(true){
    p1: Sezione Non Critica
    p2: while(B1==false){}
    p3: Sezione Critica
    p4: B1=false;
    p5: send(ok)->P(1-i)
}

--Gestione dei messaggi di Pi per i uguale a 0 o 1
case ok: B1=true;

```

- 1) L'ALGORITMO IMPLEMENTA UN MECCANISMO BASATO SUL PASSAGGIO DI UN "TOKEN" (PILLOLETTA) TRAMITE SCAMBIO DI MESSAGGI. INIZIALMENTE, SOLO IL PROCESSO P_0 HA IL TOKEN ($B_0 = \text{True}$).
UN PROCESSO PUO' SUPERARE L'ANSA ALLA RIGA P_2 ED ENTRARE IN SEZIONE CRITICA SOLO SE LA SUA VARIABILE B_i E' TRUE (OSSIA POSSIODE IL TOKEN). QUANDO TERMINA LA SC, IL PROCESSO PERDE IL TOKEN METTENDO LA PROPIA VARIABILE A FALSE E INVIA UN MESSAGGIO OK ALL'ALTRO PROCESSO. LA RICEZIONE DEL MESSAGGIO OK DA PARTE DELL'ALTRO PROCESSO IMPOSTA LA SUA VARIABILE A TRUE, CEDENDOGLI IL TOKEN.
- 2) SI E' VERIFICATA IL PERMESSO DI ENTRARE IN SC E' STRETTAMENTE LEGATO AL PASSAGGIO DEL TOKEN. PER INIZIALIZZAZIONE, C'E' UN UNO TOKEN NEL SISTEMA. PAPA CHE UN PROCESSO P_i CEDA IL TOKEN INVIANDO IL MESSAGGIO OK, DEVE SEMPRE SPECIFICARE IL PROPRIO PERMESSO ALLA RIGA P_4 . SI CONSEGUEVA E' IMPOSSIBILE AVERE $B_0 = \text{True}$ E $B_1 = \text{True}$ CONTEMPORANEAMENTE.
- 3) NO, NON E' VERIFICATA. SE IL PROCESSO P_0 ESCE DALLA SC, INVIA IL TOKEN A P_1 . SE P_1 RIMANE ALL'INFINITO NELLA SUA SEZIONE NON CRITICA SENZA MAI FARE RICHIESTA DI ENTRARE IN SC, IL TOKEN NON VARRA' MAI PIU' CEDUTO. P_0 RIMARRA' BLOCCATO PER SEMPRE AL SUO WAYPOINT.
PER RISOLVERE: BISOGLIA INTRODURRE L'IPOTESI CHE NESSUN PROCESSO SI FERMISSE PER UN TEMPO INFINITO NELLA PROPIA SEZIONE NON CRITICA. OGNI PROCESSO DEVE OBBLIGATORIAMENTE FARE IL SUO CICLO E RISPEDIRE IL TOKEN ALL'ALTRO.
- 4) ANDIAMO A ADIUSTARE IL PROCESSO NUOVO NELLA INIZIALIZZAZIONE: $B_0 = \text{True}$, $B_1 = \text{False}$, $B_2 = \text{False}$. IL MESSAGGIO NON DEVE ESSERE INVIATO A P_1 MA AL PROCESSO SUCCESSIVO.
QUINDI: $\text{SEND}(\text{OK}) \rightarrow P(i+1) \wedge .3$